

Heaps and bucket sort

5/16/25

Recall: Heaps

- Priority queue: enqueue, dequeue based on priority, peek, and size
- Efficient implementation:
 - Values stored in an array indexed from 1
 - Root stored in index 1
 - Can move thru the tree via index arithmetic:
 - left child of value in index i : index $2i$
 - right child of value in index i : index $2i+1$
 - parent of value in index i : index $i/2$ (round down)

How do we add a value to a
heap?

Upheap: letting values float up

```
void upheap(int i) {  
    //i is index that may need to move up  
    while((i > 1) && (A[i/2] > A[i])) {  
        swap A[i] and A[i/2]  
        i = i / 2;  
    }  
}
```

What heap results when 64 is added to the heap below?

1	2	3	4	5	6	7	8	9	10
4	6	5	12	8	142	7	18	215	

What heap results when 2 is added to the heap below?

1	2	3	4	5	6	7	8	9	10
4	6	5	12	8	142	7	18	215	

How do we dequeue the
minimum value?

Downheap: letting values sink down

```
void downheap(int i) {  
    while(2*i <= size) {  
        int j = 2*i;          //smaller child's index (val to replace A[i])  
        if((j < size) && (A[j] > A[j+1]))  
            j++;  
        if(A[i] <= A[j])  
            break;  
        swap A[i] and A[j]  
        i = j;  
    }  
}
```


What heap results when the minimum is removed from the heap below?

1	2	3	4	5	6	7	8	9
4	6	5	12	8	142	7	18	215

Implementing heap sort

- Can turn array into heap by running downheap on each element, starting at end (“heapifying” an array)
- As values are removed from the heap, it shrinks and frees up space in the array